



Looping Statement s in Java



Objectives:

- Identify the looping statements used in Java.
- Discuss how each looping statements worked.
- Demonstrate how looping statements work.
- Applying looping statements in creating programs.

What is a looping statement?

- A loop statement is a series of steps or sequence of statements executed repeatedly zero or more times satisfying the given condition is satisfied.
- Loops can execute a block of code as long as a specified condition is reached.
- Loops are handy because they save time, reduce errors, and they make code more readable.

Loops in Java

While Loop

~~Do/While Loop~~

For Loop



Java While Loop

- The *while loop* loops through a block of code as long as a specified condition is true:

SYNTAX:

```
while (condition) {  
    // code block to be executed  
}
```

Exam

In the example below, the code in the loop will run, over and over again, as long as a variable (i) is less than 5:

```
int i = 0;
while (i < 5)
{
    System.out.println(i);
    i++;
}
```


The Do/While Loop

- The *do/while loop* is a variant of the while loop.
- This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.

SYNTAX:

```
do {  
    // code block to be executed  
}  
while (condition);
```

EXAMP

- The example below uses a do/while loop. The loop will always be executed at least once, even if the condition is false, because the code block is executed before the condition is tested:

```
int i = 0;
do {
    System.out.println
    (i);
    i++;
}
while (i <
5):
```


Example 2:

- Create a program using do/while that displays the even numbers from 1 to 10.

Sample code:

```
int i=2;
do
    { System.out.println(i); i=i+2;
    }
while (i<=10);
```

Your turn: Try this code

```
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++;
}
```

```
int i = 0;
do {
    System.out.println(i);
    i++;
}
while (i < 5);
```

```
int i=2;
do {
    System.out.println(i);
    i=i+2;
}
while (i<=10);
```

Sample Problems

1. Create a program that displays the even numbers from 1 to 10.
2. Create a program that displays numbers divisible by 5 from 1-50.
3. Create a program that displays odd numbers from 1 to 20.
4. Create a program that accepts an input value from the user and displays the sum of the values up to 0.

Sample Input

Enter an integer: 5

Sample output

The sum is 15.

Assignment:

1. Create a program using while or do/while loop that accepts input from the user once. If the user inputs an odd number, it will display odd numbers from 1 to 10. If the user inputs an even number, even numbers from 1 to 10 will be displayed. If the user inputs 0 or any negative number, it will display invalid input.

Sample Program Output 1:

```
Enter a number: 1
Odd numbers from 1 to 10 are:
1
3
5
7
9
```

Sample Program Output 2:

```
Enter a number: 2
Even numbers from 1 to 10 are:
2
4
6
8
10
```

Sample Program Output 3:

```
Enter a number: 0
Invalid input
```

Java For Loop

- When you know exactly how many times you want to loop through a block of code, use the for loop instead of a while loop:

• SYNTAX: *i = 0* → initialization *i <= 10* → condition *if i == 10 i = i + 2* → action

```
for (statement 1; statement 2; statement 3) {  
    // code block to be executed  
}
```

SYNTAX EXPLAINED

- **Statement 1** is executed (one time) before the execution of the code block.
- **Statement 2** defines the condition for executing the code block.
- **Statement 3** is executed (every time) after the code block has been executed.
-

EXAMPLE:

Boolean expression

```
for (int i = 0; i < 5; i++)  
    { System.out.println(i);  
    }
```

- Statement 1 sets a variable before the loop starts (int i = 0).
- Statement 2 defines the condition for the loop to run (i must be less than 5). If the condition is true, the loop will start over again, if it is false, the loop will end.
- Statement 3 increases a value (i++) each time the code block in the loop has been executed.

This example will only print even values between 0 and 10:

```
for (int i = 0; i <= 10; i = i + 2)
    { System.out.println(i);
    }
```

Java Break and Continue

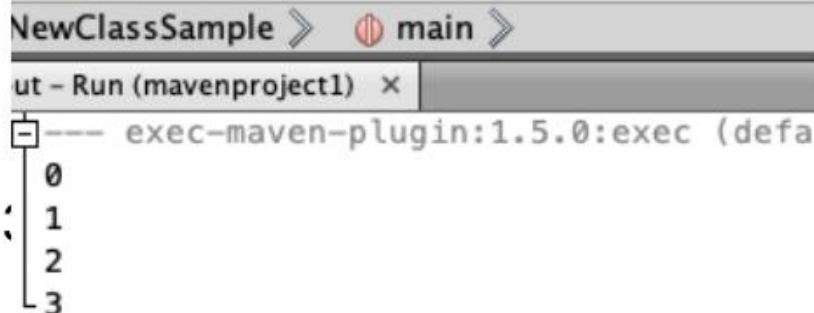
- Break is used to "jump out" of a switch statement.
- The break statement can also be used to jump out of a **loop**.

This example jumps out of the loop when i is equal to 4:

```
for (int i = 0; i < 10; i++)  
{ if (i == 4) {  
    break;  
}  
System.out.println(i);  
}
```

Expected output: 0, 1, 2, 3

```
for (int i = 0; i < 10; i++) {  
    if (i == 4) {  
        break;  
    }  
    System.out.println(i);  
}
```



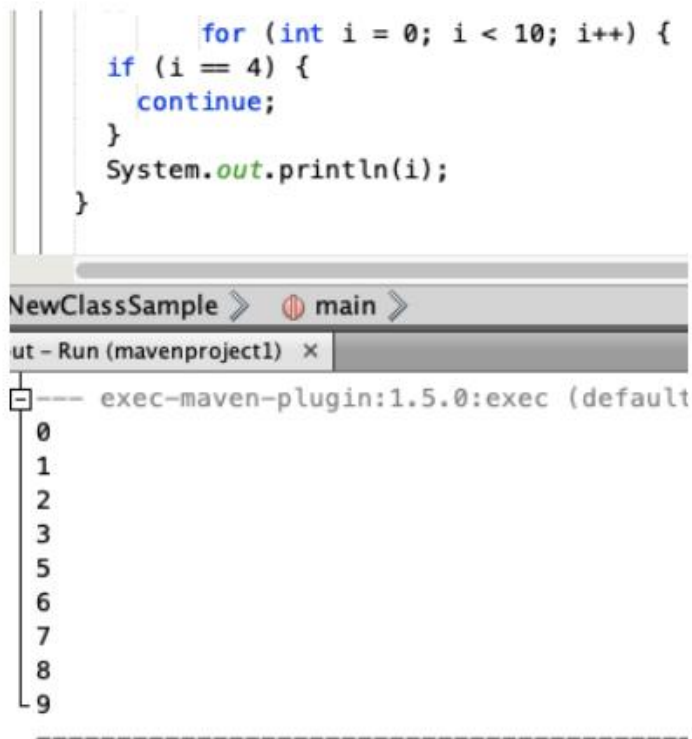
Java Continue

- The continue statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.

Examp le:

- This example skips the value of 4:

```
for (int i = 0; i < 10; i++) {  
    if (i == 4) {  
        continue;  
    }  
    System.out.println(i);  
}
```



The screenshot shows an IDE window with a Java file named `NewClassSample`. The code is a `for` loop that prints numbers from 0 to 9, but it uses `continue` to skip the iteration where `i` is 4. The output window shows the results of the program execution, which are the numbers 0 through 9, with 4 missing. The output window title is `Run - Run (mavenproject1) x` and the command used is `exec-maven-plugin:1.5.0:exec (default`.

```
for (int i = 0; i < 10; i++) {  
    if (i == 4) {  
        continue;  
    }  
    System.out.println(i);  
}
```

NewClassSample > main >

Run - Run (mavenproject1) x

exec-maven-plugin:1.5.0:exec (default

0
1
2
3
5
6
7
8
9

Break and Continue in

- You can also use break and continue while in loops:

• Break Example

```
int i = 0;
while (i < 10)
{ System.out.println(i);
  i++;
  if (i == 4) {
    break;
  }
}
```

```
int i = 0;
while (i < 10) {
    if (i == 4) {
        i++;
        break;
    }
    System.out.println(i);
    i++;
}
```

Run (mavenproject1) x

exec-maven-plugin:1.5.0:exec (d

0
1
2
3

Continue

Example

```
int i = 0;
while (i < 10) {
    if (i == 4) {
        i++;
        continue;
    }
    System.out.println(i);
    i++;
}
```

```
int i = 0;
while (i < 10) {
    if (i == 4) {
        i++;
        continue;
    }
    System.out.println(i);
    i++;
}
```

t - Run (mavenproject1) x

]--- exec-maven-plugin:1.5.0:exec (default-c'

0

1

2

3

5

6

7

8

-9

Activity:

1. Write a program that prompts the user to input an integer and then outputs the number with the digits reversed. For example, if the input is 8, the output should be 87654321.

2. Write a program that prompts the user to input an integer and based on the inputted integer, it then prints the sum of the even and odd integers.

Sample Program output:

Enter an integer: 5

Sum of numbers: 9

Handwritten red annotations showing the calculation of the sum of odd integers from 1 to 9:

- A red '9' is written above the text 'Sum of numbers: 9'.
- A red arrow points down from the '9' to the expression $1 + 3 + 5 + 7 + 9$.
- A red arrow points down from the '10' in the expression $2 + 4 + 6 + 8 + 10$ to the '9' in the expression $1 + 3 + 5 + 7 + 9$.
- A red arrow points down from the '6' in the expression $2 + 4 + 6$ to the '9' in the expression $1 + 3 + 5 + 7 + 9$.



Java break and continue



The break statement

- The break statement can also be used to jump out of a **loop**.
- This example stops the loop when i is equal to 4:

```
for (int i = 0; i < 10; i++) {  
    if (i == 4) {  
        break;  
    }  
    System.out.println(i);  
}
```

The continue statement

- The continue statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.
- This example skips the value of 4:

```
for (int i = 0; i < 10; i++) {  
    if (i == 4) {  
        continue;  
    }  
    System.out.println(i);  
}
```

Break and Continue in While Loop

- You can also use break and continue in while loops:

```
int i = 0;
while (i < 10) {
    System.out.println(i);
    i++;
    if (i == 4) {
        break;
    }
}
```

```
int i = 0;
while (i < 10) {
    if (i == 4) {
        continue;
    }
    System.out.println(i);
    i++;
}
```

Looping Exercises:

Discuss the output of the given code:

Code

```
public class Triangle {  
    public static void main(String args[])  
    {  
        int rows = 5;  
  
        for (int i = 1; i <= rows; i++) {  
            for (int j = 1; j <= i; j++) {  
                System.out.print("* ");  
            }  
            System.out.println();  
        }  
    }  
}
```

Output

```
*  
* *  
* * *  
* * * *  
* * * * *
```

Looping Exercises: Discuss the output of the given code.

```
public class SumEvenNum {  
    public static void main(String args[])  
    {  
        int sum = 0;  
        int number = 2;  
  
        do {  
            sum += number;  
            number += 2;  
        } while (number <= 20);  
  
        System.out.println("Sum of even numbers from 1 to 20 is: " + sum);  
    }  
}
```

Output

```
exec-maven-plugin:1.5.0:exec (default-  
Sum of even numbers from 1 to 20 is: 110
```

Looping Exercises

Code

```
public class MultiplicationTable {  
    public static void main (String[] args)  
    {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Multiplication table for " + i + ":");  
            for (int j = 1; j <= 10; j++) {  
                System.out.println(i + " * " + j + " = " + (i * j));  
            }  
            System.out.println();  
        }  
    }  
}
```

Looping

Multiplication table for 1:

1 * 1 = 1
1 * 2 = 2
1 * 3 = 3
1 * 4 = 4
1 * 5 = 5
1 * 6 = 6
1 * 7 = 7
1 * 8 = 8
1 * 9 = 9
1 * 10 = 10

Multiplication table for 2:

2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8
2 * 5 = 10
2 * 6 = 12
2 * 7 = 14
2 * 8 = 16
2 * 9 = 18
2 * 10 = 20

Output

Multiplication table for 3:

3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30

Multiplication table for 4:

4 * 1 = 4
4 * 2 = 8
4 * 3 = 12
4 * 4 = 16
4 * 5 = 20
4 * 6 = 24
4 * 7 = 28
4 * 8 = 32
4 * 9 = 36
4 * 10 = 40

Multiplication table for 5:

5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50